

AutoEncoder

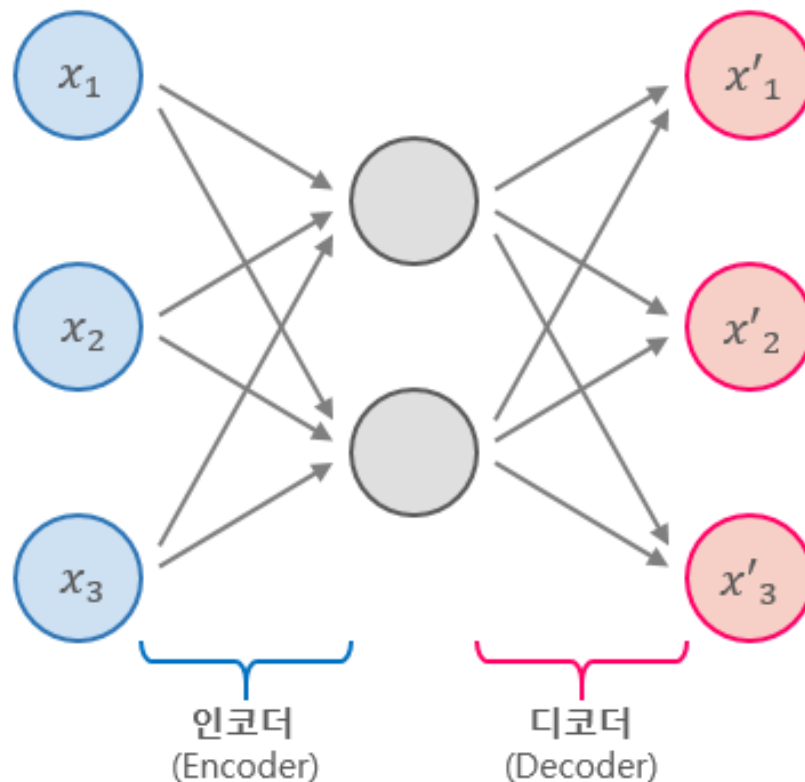
Fall 2025

School of IT Convergence

Prof. Dongsup Jin

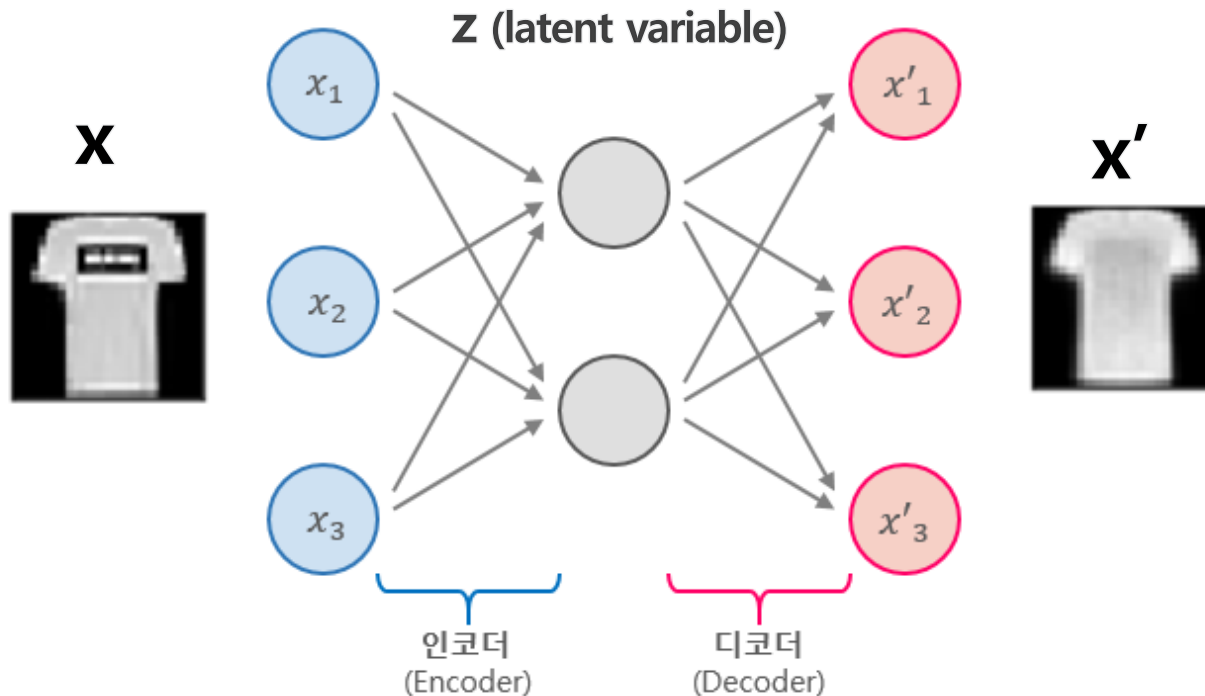
AutoEncoder (1)

- Self-supervised learning
 - A neural network that extract features without labels



AutoEncoder (2)

- Features
 - # dim of Input x == # dim of Output x'
 - The dimension decreases towards the middle
 - The features of the input are trained to be compressed



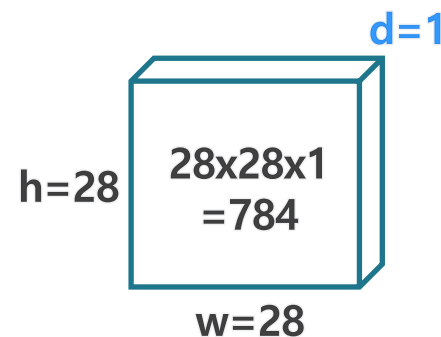
AutoEncoder (3)

- Latent variable (z)
 - Data represented in high density in low dimensions
 - Meaning
 - Information organization has priority
 - (Ex) Background color is not a very important factor in portrait photography
 - The process of discarding less important features
 - Data processing that leaves only important information
- It is mainly used to reduce the dimensionality of complex nonlinear data

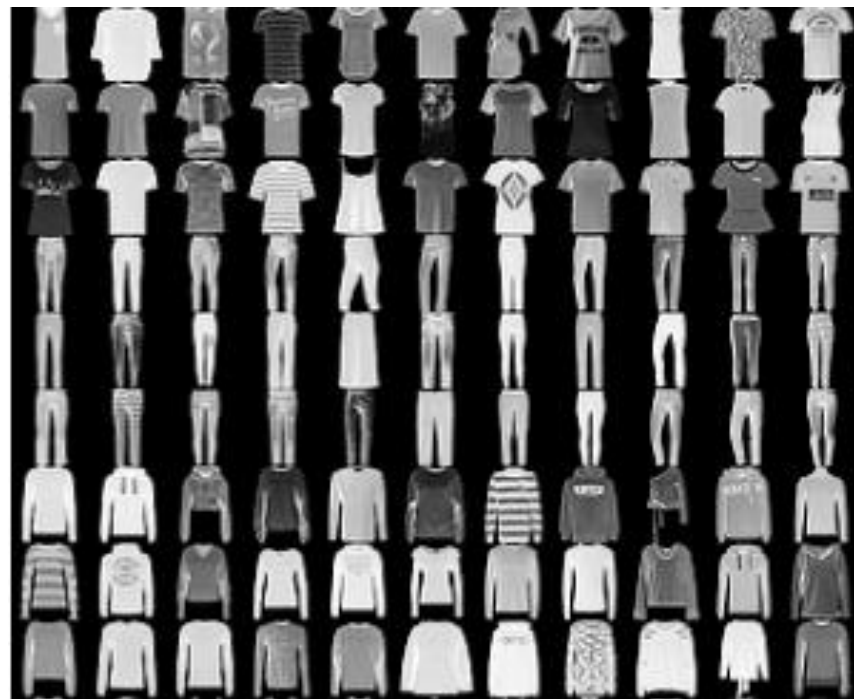
Revisit - Fashion MNIST

- Fashion-MNIST

- A dataset of Zalando's article images
- <https://github.com/zalandoresearch/fashion-mnist>
- Total 70,000 images
 - Training set: 60,000 images
 - Test set: 10,000 images
- Image size: **28x28 grayscale image**
- Label: 10 classes



Label	Description		
0	T-shirt/top	5	Sandal
1	Trouser	6	Shirt
2	Pullover	7	Sneaker
3	Dress	8	Bag
4	Coat	9	Ankle boot



Dataset images

AutoEncoder Implementation (1)

1. Import several packages and modules

```
import torch
import torchvision
import torch.nn.functional as F
from torch import nn, optim
from torchvision import transforms, datasets

import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from matplotlib import cm
import numpy as np
```

2. Set device (cuda or cpu) and hyper parameters

```
EPOCH = 10
BATCH_SIZE = 64
USE_CUDA = torch.cuda.is_available()
DEVICE = torch.device("cuda" if USE_CUDA else "cpu")
print("Using Device:", DEVICE)
```

AutoEncoder Implementation (2)

3. Load dataset (Fashion MNIST)

```
trainset = datasets.FashionMNIST(
    root      = './.data/',
    train     = True,
    download  = True,
    transform = transforms.ToTensor()
)
train_loader = torch.utils.data.DataLoader(
    dataset    = trainset,
    batch_size = BATCH_SIZE,
    shuffle    = True,
    num_workers = 2
)
```

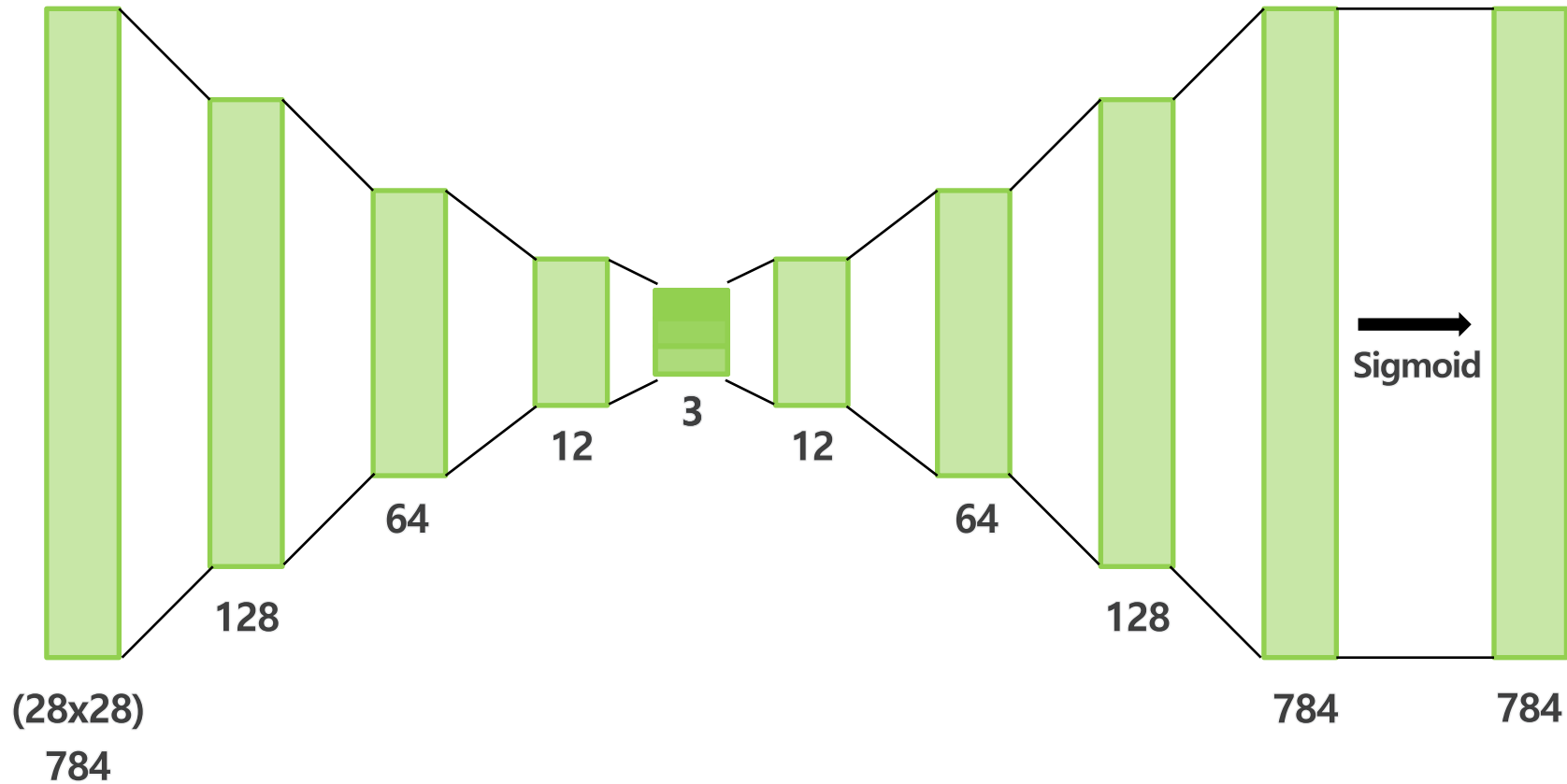
4. Define AutoEncoder Class

```
class Autoencoder(nn.Module):
    def __init__(self):
        super(Autoencoder, self).__init__()

        self.encoder = nn.Sequential(
            nn.Linear(28*28, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 12),
            nn.ReLU(),
            nn.Linear(12, 3),
        )
        self.decoder = nn.Sequential(
            nn.Linear(3, 12),
            nn.ReLU(),
            nn.Linear(12, 64),
            nn.ReLU(),
            nn.Linear(64, 128),
            nn.ReLU(),
            nn.Linear(128, 28*28),
            nn.Sigmoid(),
        )

    def forward(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return encoded, decoded
```

AutoEncoder Implementation (3)



AutoEncoder Implementation (4)

5. Load model and Set optimization

```
autoencoder = Autoencoder().to(DEVICE)
optimizer = torch.optim.Adam(autoencoder.parameters(), lr=0.005)
criterion = nn.MSELoss()

view_data = trainset.data[:5].view(-1, 28*28)
view_data = view_data.type(torch.FloatTensor)/255.
```

6. Define train module

```
def train(autoencoder, train_loader):
    autoencoder.train()
    for step, (x, label) in enumerate(train_loader):
        x = x.view(-1, 28*28).to(DEVICE)
        y = x.view(-1, 28*28).to(DEVICE)
        label = label.to(DEVICE)

        encoded, decoded = autoencoder(x)

        loss = criterion(decoded, y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

AutoEncoder Implementation (5)

7. Test

```
for epoch in range(1, EPOCH+1):
    train(autoencoder, train_loader)

    # 디코더에서 나온 이미지를 시각화 하기 (두번째 열)
    test_x = view_data.to(DEVICE)
    _, decoded_data = autoencoder(test_x)

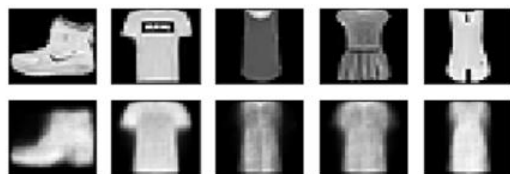
    # 원본과 디코딩 결과 비교해보기
    f, a = plt.subplots(2, 5, figsize=(5, 2))
    print("[Epoch {}].format(epoch))
    for i in range(5):
        img = np.reshape(view_data.data.numpy()[i],(28, 28))
        a[0][i].imshow(img, cmap='gray')
        a[0][i].set_xticks(()); a[0][i].set_yticks(())

    for i in range(5):
        img = np.reshape(decoded_data.to("cpu").data.numpy()[i], (28, 28))
        a[1][i].imshow(img, cmap='gray')
        a[1][i].set_xticks(()); a[1][i].set_yticks(())
    plt.show()
```

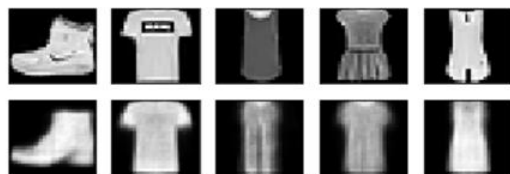
AutoEncoder Implementation (6)

- AE's train process results

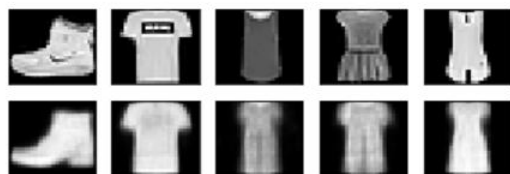
[Epoch 1]



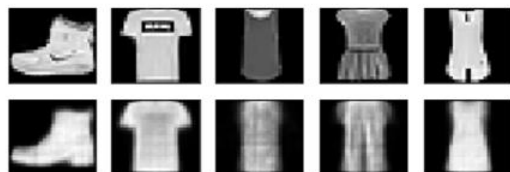
[Epoch 2]



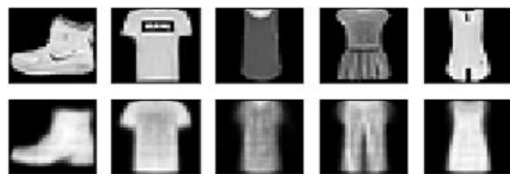
[Epoch 3]



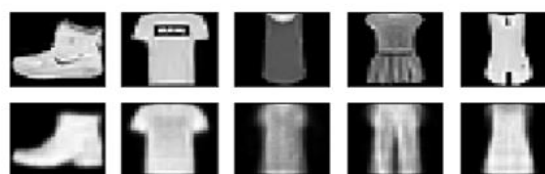
[Epoch 4]



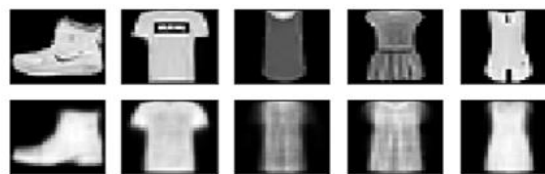
[Epoch 5]



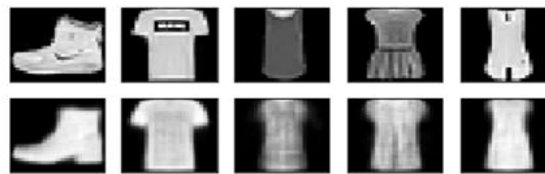
[Epoch 6]



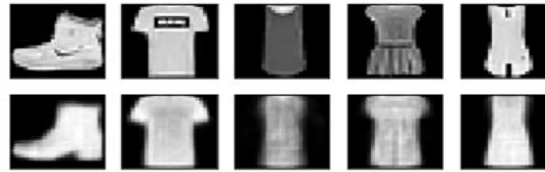
[Epoch 7]



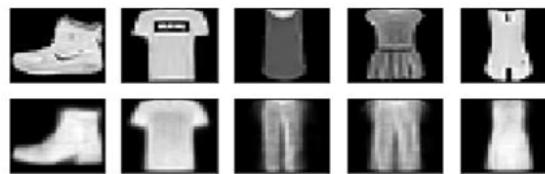
[Epoch 8]



[Epoch 9]

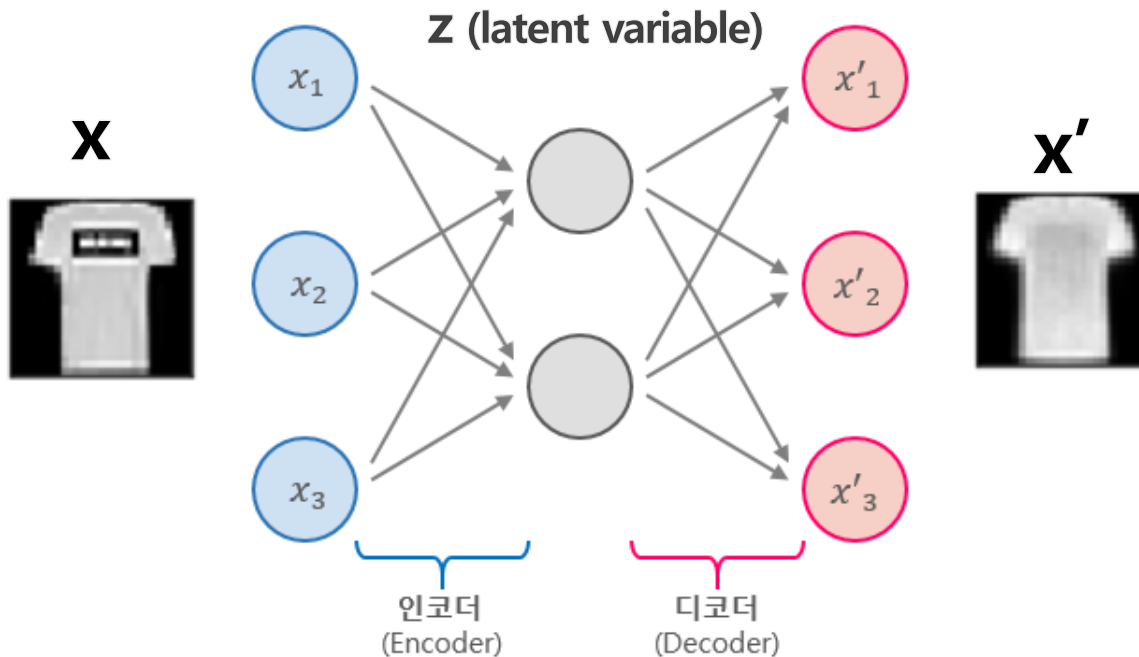


[Epoch 10]



Latent Variable (1)

- Latent variable (잠재 변수)
 - A variable that are measured indirectly using observable variables
 - The hidden or unobserved elements



Latent Variable (2)

- (Ex) Latent variable 3D plot
 - (Dataset) Fashion MNIST

8. Prepare 200 input original images from the training dataset, and then encode the images

```
view_data = trainset.data[:200].view(-1, 28*28)
view_data = view_data.type(torch.FloatTensor)/255.
test_x = view_data.to(DEVICE)
encoded_data, _ = autoencoder(test_x)
encoded_data = encoded_data.to("cpu")
```

```
CLASSES = {
    0: 'T-shirt/top',
    1: 'Trouser',
    2: 'Pullover',
    3: 'Dress',
    4: 'Coat',
    5: 'Sandal',
    6: 'Shirt',
    7: 'Sneaker',
    8: 'Bag',
    9: 'Ankle boot'
}
```

Latent Variable (3)

9. Plot the encoded images in 3D space

```
fig = plt.figure(figsize=(10,8))
ax = Axes3D(fig)

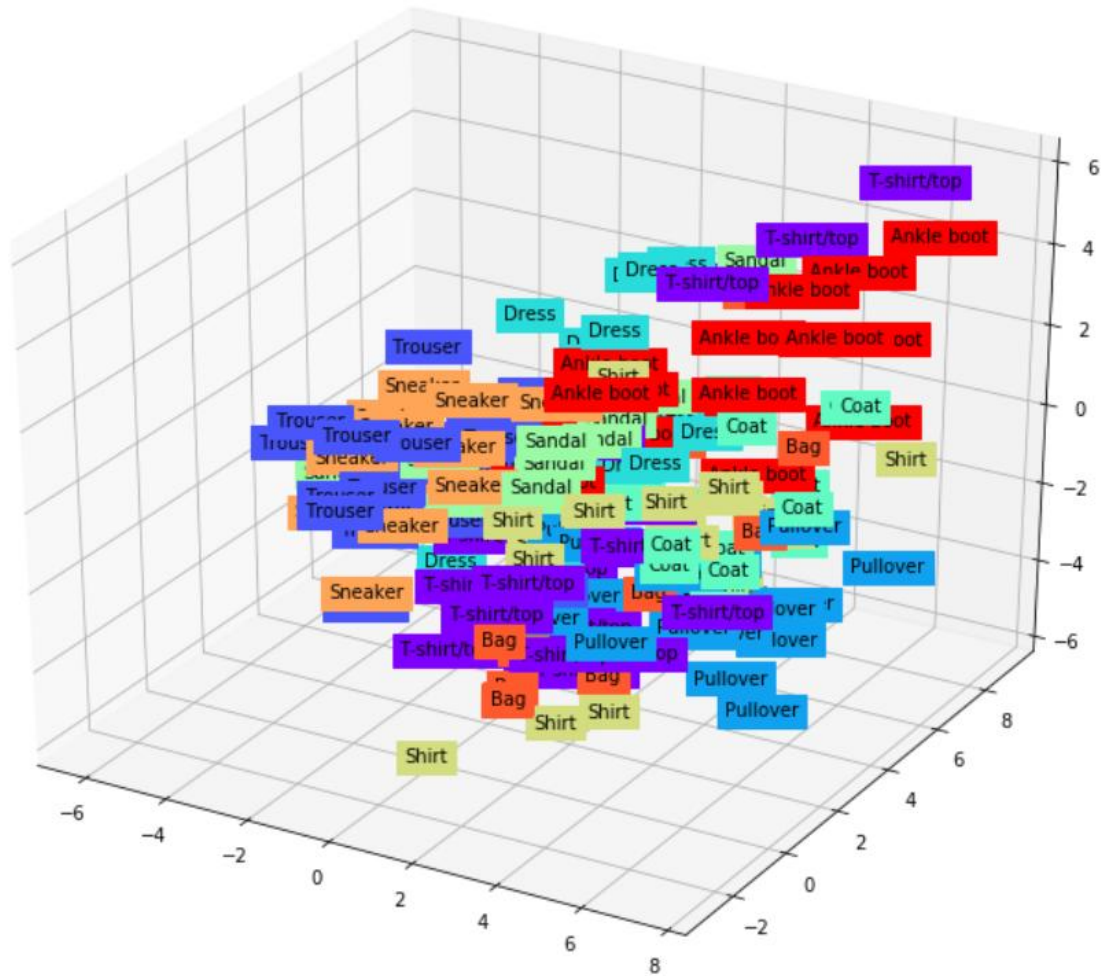
X = encoded_data.data[:, 0].numpy()
Y = encoded_data.data[:, 1].numpy()
Z = encoded_data.data[:, 2].numpy()

labels = trainset.targets[:200].numpy()

for x, y, z, s in zip(X, Y, Z, labels):
    name = CLASSES[s]
    color = cm.rainbow(int(255*s/9))
    ax.text(x, y, z, name, backgroundcolor=color)

ax.set_xlim(X.min(), X.max())
ax.set_ylim(Y.min(), Y.max())
ax.set_zlim(Z.min(), Z.max())
plt.show()
```

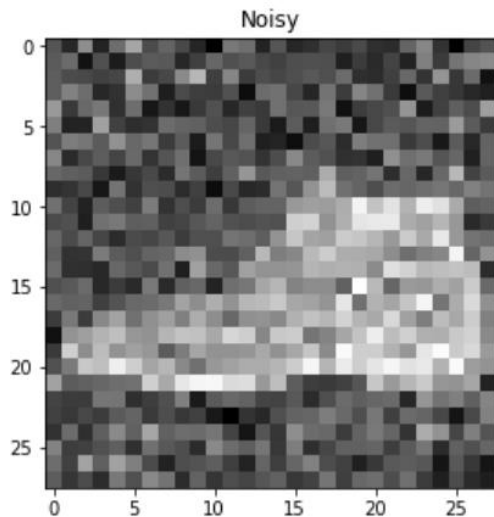
Latent Variable (4)



Visualization of the encoded image in 3D space

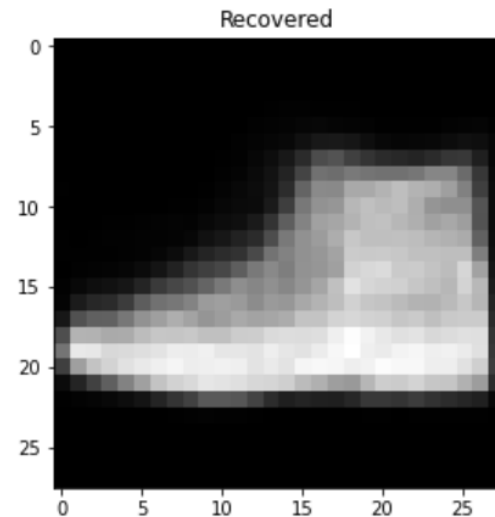
Denoising AutoEncoder (1)

- Denoising using AutoEncoder
 - (AE's feature) Remove relatively less important features
 - Remove noises using AutoEncoder



Noisy image

→
Denoising



Recovered image

Denoising AutoEncoder (2)

1. Define 'Add_noise' module that Add random noises to the training input images

```
def add_noise(img):  
    noise = torch.randn(img.size()) * 0.2  
    noisy_img = img + noise  
    return noisy_img
```

2. Define train module

```
def train(autoencoder, train_loader):  
    autoencoder.train()  
    avg_loss = 0  
    for step, (x, label) in enumerate(train_loader):  
        noisy_x = add_noise(x) # 입력에 노이즈 더하기  
        noisy_x = noisy_x.view(-1, 28*28).to(DEVICE)  
        y = x.view(-1, 28*28).to(DEVICE)  
  
        label = label.to(DEVICE)  
        encoded, decoded = autoencoder(noisy_x)  
  
        loss = criterion(decoded, y)  
        optimizer.zero_grad()  
        loss.backward()  
        optimizer.step()  
  
        avg_loss += loss.item()  
    return avg_loss / len(train_loader)
```

Denoising AutoEncoder (3)

3. Check Loss function values per epoch step

```
def add_noise(img):  
    noise = torch.randn(img.size()) * 0.2  
    noisy_img = img + noise  
    return noisy_img
```



```
[Epoch 1] loss:0.03753504549254423  
[Epoch 2] loss:0.026430994466836773  
[Epoch 3] loss:0.02479601246731749  
[Epoch 4] loss:0.02395139931480704  
[Epoch 5] loss:0.02353472688765541  
[Epoch 6] loss:0.023220639165514693  
[Epoch 7] loss:0.0229087375692213  
[Epoch 8] loss:0.022784441957738735  
[Epoch 9] loss:0.022587720000905902  
[Epoch 10] loss:0.02250293003065563
```

Denoising AutoEncoder (4)

4. Visualize denoising

```
testset = datasets.FashionMNIST(
    root      = './.data/',
    train     = False,
    download  = True,
    transform = transforms.ToTensor()
)
sample_data = testset.data[0].view(-1, 28*28)
sample_data = sample_data.type(torch.FloatTensor)/255.

original_x = sample_data[0]
noisy_x = add_noise(original_x).to(DEVICE)
_, recovered_x = autoencoder(noisy_x)
```

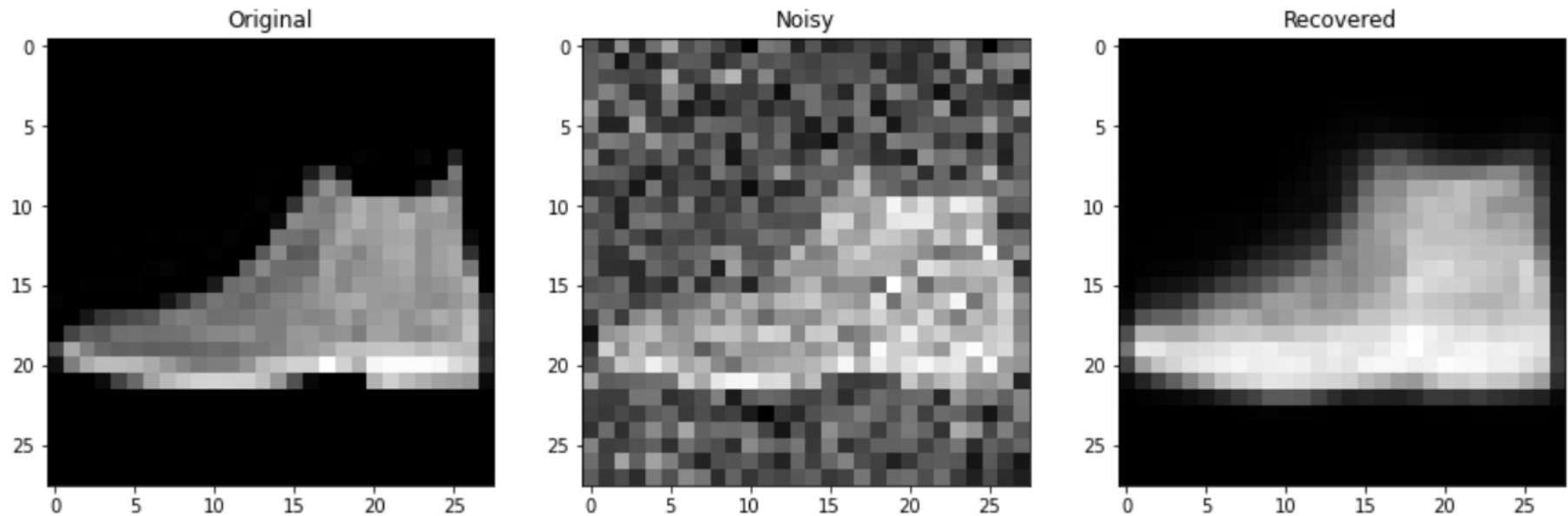
```
f, a = plt.subplots(1, 3, figsize=(15, 15))

original_img = np.reshape(original_x.to("cpu").data.numpy(), (28, 28))
noisy_img = np.reshape(noisy_x.to("cpu").data.numpy(), (28, 28))
recovered_img = np.reshape(recovered_x.to("cpu").data.numpy(), (28, 28))

a[0].set_title('Original')
a[0].imshow(original_img, cmap='gray')
a[1].set_title('Noisy')
a[1].imshow(noisy_img, cmap='gray')
a[2].set_title('Recovered')
a[2].imshow(recovered_img, cmap='gray')

plt.show()
```

Denoising AutoEncoder (5)



A denoising example using AutoEncoder

Q & A
